

CONFIDENTIAL & PROPRIETARY

MAHARAJA INSTITUTE OF TECHNOLOGY MYSORE

Department of Computer Science and Engineering

ACADEMIC OPS PLATFORM

DETAILED PROJECT REPORT

A governance-first, multi-tenant academic operations system for engineering colleges. Designed to digitise and automate the complete academic lifecycle from student admissions to mentorship tracking, course-batch assignment, internal assessment, and semester-end results.

REPORT DATE

July 14, 2026

VERSION

1.0.0 (Production-Ready)

CLASSIFICATION

Proprietary & Confidential

TECH STACK

Next.js 16 | Express 5 | Prisma 7 | PostgreSQL 16

Table of Contents

1.0	Executive Summary	3
2.0	Project Overview	4
3.0	Software Description & Feature Modules	5
4.0	System Architecture	7
5.0	Technology Stack Analysis	9
6.0	Core Operations & Workflows	10
7.0	Database and Data Management	12
8.0	Security Architecture	14
9.0	Scalability and Performance	16
10.0	Deployment and DevOps	17
11.0	Project Structure	19
12.0	Compliance and Audit	20
13.0	Key Benefits	21
14.0	Contributing	21
15.0	Conclusion	22

Note: This document is compiled directly from the production-hardened source code, schema migrations, and technical configurations of the Academic Ops Platform. All details reflect the actual system architecture as of July 2026.

1.0 Executive Summary

Academic Ops Platform is an enterprise-grade, multi-tenant Software-as-a-Service (SaaS) web application purpose-built to digitise, automate, and secure the academic operations of engineering colleges. Traditionally, academic management relies on fragmented spreadsheet files, manual data entries, and paper forms. These manual workflows are highly prone to data leakage, lack auditability, and introduce severe administrative delays.

By consolidating the entire academic pipeline — from admission intake and role-based user management to daily attendance, internal assessment computation, and Controller of Examinations (COE) results publishing — Academic Ops provides a zero-trust, governance-first operational backbone. Data is strictly isolated at the tenant level, ensuring complete privacy between individual institutions.

Dimension	Detail
Platform Type	Multi-Tenant SaaS with dynamic slug-based routing
Core Modules	Admissions, Academics, Attendance, Internal Assessment, COE Exam, Mentorship, Chat
Target Users	Students, Parents, Teachers, Department Admins, COE, Clerks, Super Admins (10+ roles)
Isolation Model	Express-level tenant middleware with tenantId-scoped database queries
Security Stack	JWT access/refresh tokens, Argon2 hashing, Zod validations, Helmet, XSS filters
Infrastructure	Render (API), Netlify (Client), Supabase PostgreSQL, Upstash Redis, BullMQ (Workers)

Key Strategic Benefit: Enforces Separation of Duties. Faculty enter marks, Mentors review and verify, Department Admins lock configuration, Clerks upload final entries, COE approves, and the Super Admin manages systemic parameters — all within a tamper-resistant environment backed by detailed audit logs.

2.0 Project Overview

2.1 Background and Problem Statement

Engineering colleges in India manage highly complex academic workflows. These involve thousands of students across multiple programs (e.g., B.Tech, M.Tech, MBA), departments (e.g., CSE, ECE, ISE), and semesters. Typically, these operations are governed by disconnected software systems or basic spreadsheets. Critical tasks — such as tracking 85% attendance compliance, calculating best-of-N internal assessments, managing first-year cycle swaps (Physics/Chemistry cycle allocations), and entering semester exam marks — are done manually.

This manual administration leads to recurring problems: lack of historical audit trails (who modified a student's marks?), unauthorized access to grading systems, delays in result compilation, and the inability to communicate critical updates (like parent notifications for student absences) in real time.

2.2 Proposed Solution

Academic Ops replaces legacy processes with a centralized, secure web application. It enforces a strict, hierarchical permission model that ensures no grade can be modified without a formal approval request, and no tenant can view another tenant's records. It integrates real-time communications, automated calculations, and a structured audit log showing exact old and new values for every database modification.

2.3 Vision and Scope

- **Multi-Tenancy:** Serve multiple affiliated colleges from a single deployment while maintaining complete data isolation.
- **Separation of Duties:** Differentiate administrative access across 10 distinct user roles.
- **Academic Integrity:** Lock semester modules, courses, and attendance sheets once finalized.
- **Real-time Insights:** Provide instant dashboard metrics to department heads and college management.
- **Parental Involvement:** Enable parents to monitor their child's attendance and marks, and chat directly with their assigned mentor.

3.0 Software Description

3.1 Core Product Capabilities

The Academic Ops Platform is designed with modularity, allowing institutions to selectively deploy features. The application leverages Next.js 16 to provide a highly interactive, responsive frontend that communicates with an Express 5 REST API. Data integrity is guaranteed via Prisma ORM schemas, and email delivery is managed through a background BullMQ Redis queue.

3.2 Role-Based Dashboards

Access control is enforced both on the client (Next.js route guards) and server (Express RBAC middleware). The system defines 10 specific tenant-level roles plus a global developer portal:

Role	Scope	Key Responsibilities
Developer	Global System	Tenant provisioning, system error tracking, email delivery logs, multi-tenant analytics.
Super Admin	Institution-wide	User management, department creation, program duration configuration, batch progression.
COE	Institution-wide	Course catalogue locks, exam marks approval, result publication, permanent USN assignment.
Dept Admin	Departmental	Course allocation, IA configuration, section locks, mentor assignment, audit log monitoring.
Admissions Admin	Institution-wide	Admission form builder configuration, applicant review, temporary USN allocation.

Security Tip: A database trigger explicitly blocks direct updates to the `role` column in the `User` table, preventing role escalation attacks by malicious actors bypass-attempting API routes.

3.0 Software Description (Cont.)

Role	Scope	Key Responsibilities
Admin Clerk	Assigned task	Data entry for admissions under the supervision of the Admissions Admin.
Clerk	Assigned task	Semester-end marks entry, revaluation details uploading via learning Excel parser.
FY Coordinator	First-Year Cycle	Physics and Chemistry cycle allocations, first-year student section assignments.
Teacher	Assigned Subjects	Attendance marking, assignment management, IA marks entry, mentor observation cards.
Student	Own Data Only	View attendance history, IA scores, final results, personal profile, and timetables.
Parent	Linked Student	View linked child's daily attendance, academic marks, and chat directly with their mentor.

3.3 Additional Platform Features

- **Dark / Light Theme:** Per-user UI aesthetic preference, persisted to the database profile.
- **Responsive Layout:** Fully mobile-first design targeting desktop, tablet, and mobile screens.
- **Excel Parsing with Self-Learning:** Automates student and mark uploads while dynamically correcting column headers (e.g. "USN" mapped to "University Register Number") based on past user decisions.
- **Real-time Parent-Mentor Chat:** Supports instant communication with escalation triggers to Department Admins for unresolved student issues.

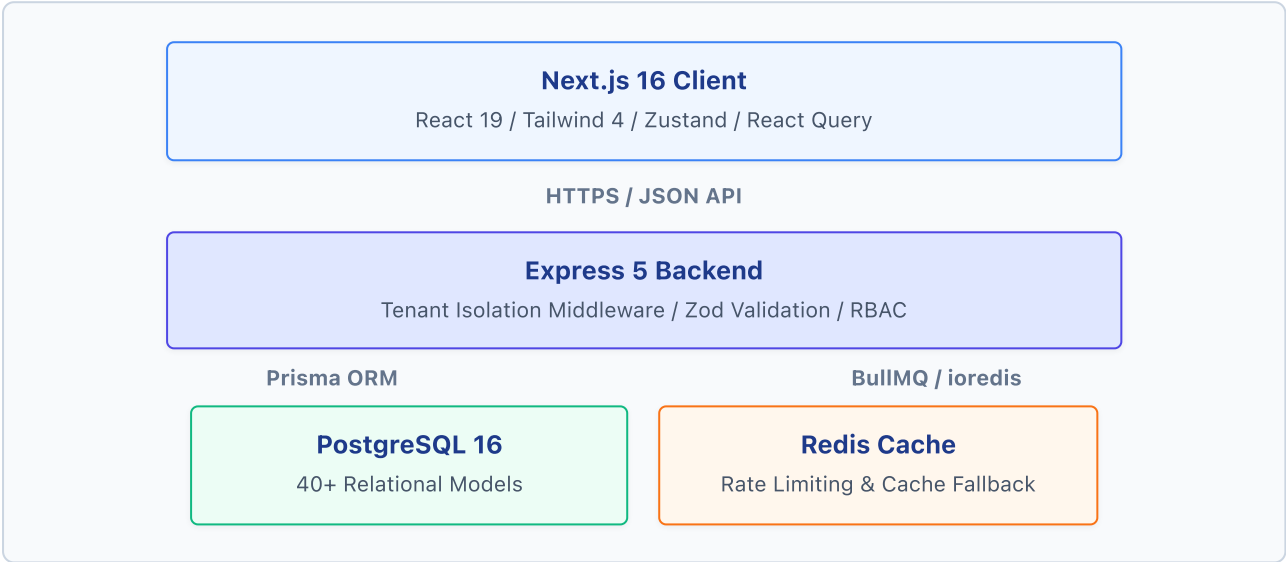
3.4 Developer and Support Portal

An integrated issue tracking system lets users report UI bugs, accessibility issues, or wrong data directly from their top navigation bar. The platform automatically attaches client metadata — including browser details, OS version, screen resolution, and URL context — to help administrators rapidly diagnose issues.

4.0 System Architecture

4.1 High-Level Architecture

Academic Ops utilizes a modern decoupled full-stack architecture. The Next.js 16 frontend operates as a client-side single-page application (using dynamic SSR where necessary). The Express 5 backend handles all business logic, data sanitization, and transaction management, exposing a RESTful JSON API.



4.2 Multi-Tenant Data Isolation

Unlike shared-schema databases that rely on database Row Level Security (RLS) alone — which can degrade query performance with complex relations — Academic Ops implements a strict application-level isolation model:

- Every tenant (college) is provisioned with a unique, immutable UUID (`tenantId`) and a custom slug (e.g. `/apply/mit-mysore`).
- A centralized **Tenant Middleware** interceptor runs on all non-public API endpoints. It extracts the active tenant ID from the JWT payload or requested route and binds it to the execution context.
- All Prisma database queries programmatically append the predicate `where: { tenantId }` , guaranteeing that no user can read, update, or delete data belonging to another college.

4.0 System Architecture (Cont.)

4.3 Services and Middleware Layer

The backend architecture organizes code into distinct layers: Routes (HTTP endpoints), Middleware (security and isolation), Services (business logic, transactions), and Repositories (database access).

Middleware File	Execution Hook	Functional Purpose
tenant.middleware.ts	Pre-Route Handler	Extracts and validates the tenant context from the request header or URL slug; scopes Prisma client calls.
auth.middleware.ts	Pre-Route Handler	Validates the JWT access token, checks the token version to block revoked sessions, and attaches the active user profile.
rate-limiters.ts	Pre-Route Handler	Applies Redis-backed rate limits. Critical endpoints (like login/payments) are limited to 5 requests per minute.
sanitize.middleware.ts	Pre-Route Handler	Sanitizes all incoming request body fields using the <code>xss</code> library to prevent script injections.
cache.middleware.ts	Route Handler / Cache	Checks Redis for cached JSON responses; serves read-only routes immediately without hitting PostgreSQL.
error.middleware.ts	Post-Route Handler	Catches database constraint violations or runtime errors, logs them structured via Pino, and returns standard error codes.

BullMQ Worker Process: A dedicated background job worker runs alongside the main Express thread. It subscribes to an email and SMS queue hosted in Redis, executing asynchronous email dispatches (via Brevo API or Gmail SMTP) and absence alerts (via Fast2SMS). This keeps HTTP response times under 50ms, even during bulk student operations.

5.0 Technology Stack Analysis

Layer	Technology Option	Version	Purpose / Choice Rationale
Frontend (client/)	Next.js (App Router)	16.1.6	React framework supporting optimized SSR, routing, and fast load times.
	Tailwind CSS	4.2.x	Utility-first styles with compiled CSS performance and dynamic variables.
	Zustand	5.x	Lightweight client state manager for auth token persistence and theme states.
	TanStack Query (v5)	5.x	Server state management, background caching, and automatic pagination refetches.
Backend (server/)	Express	5.2.x	Lightweight REST API framework with native promise routing for async/await support.
	Prisma ORM	7.4.x	Type-safe database mapping, automatic migrations, and transaction controls.
	Zod	4.x	Runtime schema validation for all API inputs and environment variables.
	Argon2	0.44	Modern password hashing algorithm, highly resistant to GPU brute-force.
	ExcelJS / BullMQ	4.x / 5.x	Excel parsing for bulk marks uploading; Redis queue for background tasks.
Infrastructure	PostgreSQL	16	Primary relational database with transactional integrity and indexing.
	PgBouncer	1.x	Connection pooler executing in transaction mode to manage database loads.
	Redis (Upstash)	7	Caching, token blocklisting, rate limiting, and queue messaging.
	Render / Netlify	—	API server environment (Render) and static Next.js build distribution (Netlify).

6.0 Core Operations & Workflows

6.1 Student Admission & USN Allocation Pipeline

The admission module is a dynamic, multi-step pipeline engineered to collect applicant data and transition it into active academic profiles. The process operates under strict role separation:

- 1 Form Builder:** Admissions Admin defines custom, tenant-specific field configurations (e.g. required documents, income certificates, lateral entry tags) using a drag-and-drop schema editor.
- 2 Application:** Prospective students fill the multi-page public admission form located at `/apply/[tenant-slug]`. Documents are uploaded directly to secure storage (S3/R2).
- 3 Review & Entry:** Admin Clerks review application details. If valid, they log entries into the system database under a `PENDING_APPROVAL` status.
- 4 Department Closure:** Once the admission window ends, the Admissions Admin triggers "Department Closure". This locks the cohort and auto-allocates a **Temporary USN** (e.g., `TEMP-2026-CSE-045`).
- 5 COE Finalisation:** The Controller of Examinations (COE) reviews the closed cohort and assigns the permanent **University Seat Number (USN)** (e.g. `4MT26CS045`), automatically generating student logins.

6.2 Academic Progression and Swaps

First-Year Cycle Swapping: In Indian engineering colleges, first-year students are split into two cycles: the Physics Cycle and the Chemistry Cycle. The platform incorporates a specialized First Year Coordinator (FYC) role to manage this lifecycle. At the end of Semester 1, the FYC triggers a batch swap. The system automatically shifts students to their respective opposite cycle departments, re-allocates subject enrollments, and resets attendance sheets for Semester 2 while archiving Semester 1 records.

6.0 Core Operations & Workflows (Cont.)

6.3 Attendance Lifecycle & Edit Request Workflow

Attendance tracking enforces academic compliance with strict modification guards:

- **Marking Attendance:** Teachers mark student status (PRESENT, ABSENT, LATE, EXCUSED) on a date-wise and subject-wise register.
- **Absence Alerts:** Saving an absence triggers a background job that dispatches a real-time SMS to parents via the Fast2SMS API integration if enabled.
- **Oversight & Locking:** Department Admins review departmental attendance sheets. At the end of each month, the Admin locks the records, preventing direct edits.
- **Edit Request Workflow:** If a teacher needs to correct an entry on a locked sheet, they submit a formal request. The Department Admin reviews the requested change and applies it, logging the entire transaction to the audit logs.

6.4 Internal Assessment (IA) Marks Computation

The system automates the calculation of internal assessment scores according to university rules:

1. **IA Configuration:** Department Admins set course assessment structures (e.g. 3 Internals, 2 Assignments, best-of-2 internals, maximum marks 50).
2. **Marks Entry:** Teachers enter scores for individual IA components. The frontend visualizes student performance with bar charts.
3. **Best-of-N Automatic Selection:** The system automatically extracts the highest scores (best-of-N internals) and computes the weighted total score.
4. **Verification Pipeline:** The Mentor checks the computed scores of their mentees. The Department Admin verifies all departmental marks, and the COE locks the final internal sheets.

6.5 COE Semester Results Upload and Approval Pipeline

The semester-end grading pipeline utilizes a strict validation process:

- **Clerk Upload:** The Clerk uploads student external marks via an Excel spreadsheet.
- **Self-Learning Column Correction:** The Excel parser automatically resolves structural mismatches (e.g., matching "Uni Seat No" to the USN schema column) based on historical correction records.
- **COE Approval:** The COE reviews the parsed marks against internal assessment scores, executes pass/fail formulas, and posts the results.

7.0 Database and Data Management

7.1 Schema Architecture

The database schema consists of over 40 structured tables defined in a single, cohesive Prisma schema (`schema.prisma`). Relational integrity is enforced using foreign key constraints and composite indexes.

7.2 Core Model Tables

Model Name	Purpose	Primary & Foreign Keys
Developer	Global system developer accounts.	id (PK), email (Unique)
Tenant	Colleges/Institutions using the system.	id (PK), slug (Unique), brandingInfo
User	Consolidated users for authentication.	id (PK), email (Unique), tenantId (FK), role
StudentProfile	Student details, USN, roll number.	id (PK), userId (FK), tenantId (FK), sectionId (FK)
TeacherProfile	Faculty details, designation, code.	id (PK), userId (FK), tenantId (FK), departmentId (FK)
ParentProfile	Linked parent contact info.	id (PK), userId (FK), tenantId (FK), studentId (FK)

7.3 Academic Model Tables

Model Name	Purpose	Primary & Foreign Keys
Department	Academic divisions (CSE, ISE, ECE).	id (PK), tenantId (FK), cycleDeptFlag
Program	Degrees offered (B.Tech, M.Tech).	id (PK), tenantId (FK), durationYears
Course	Generic courses (e.g. Operating Systems).	id (PK), tenantId (FK), credits, maxIA, maxEA
Batch	Student entry cohort (e.g. 2024-2028).	id (PK), tenantId (FK), isGraduated
Semester	Academic terms with status (ACTIVE/CLOSED).	id (PK), tenantId (FK), status
Section	Classroom divisions (Section A, B).	id (PK), batchId (FK), departmentId (FK)
Subject	Active course in a specific semester.	id (PK), courseId (FK), semesterId (FK), sectionId (FK)

7.0 Database and Data Management (Cont.)

7.4 Operational and Audit Model Tables

Model Name	Purpose	Primary & Foreign Keys
Attendance	Daily attendance log records.	id (PK), studentId (FK), subjectId (FK), status, date
Marks	Course component marks details.	id (PK), studentId (FK), subjectId (FK), iaScore, assignmentScore
InternalMarksDetail	Calculated internals with approvals.	id (PK), studentId (FK), subjectId (FK), mentorApproved, finalScore
SemesterEndMarks	Clerk uploaded exam scores.	id (PK), studentId (FK), courseId (FK), externalScore, isPosted
Result	Final calculated CGPA, grades.	id (PK), studentId (FK), semesterId (FK), sgpa, cgpa, status
AuditLog	Immutable system change history.	id (PK), userId (FK), action, oldValue, newValue, ipAddress
SystemError	Centralized developer error tracking.	id (PK), tenantId (FK), errorCode, stackTrace, severity

7.5 Database Scaling and Projections

Table Name	Frequency / Scope	Average Rows (1 Tenant)	Estimated Row Size	Total Storage
User / Profiles	One record per user	~2,500 rows	0.8 KB	2.0 MB
Enrollment	~8 subjects per student	~16,000 rows	0.4 KB	6.4 MB
Attendance	Daily entries per subject	~480,000 rows / sem	0.2 KB	96.0 MB
Marks / IA Details	Marks per subject per sem	~16,000 rows / sem	0.5 KB	8.0 MB
AuditLog	Continuous log generation	~300,000 rows / year	1.2 KB	360.0 MB
Total Scoped Data	Active Database Size	~514,500 rows	—	~472.4 MB

*Note: Projections based on 100 working days per semester with an average of 4 lectures marked daily per student. Strategic composite indexing on (tenantId, studentId, subjectId) prevents database performance degradation.

8.0 Security Architecture

8.1 Authentication and Session Security

Academic Ops follows a zero-trust architecture. It uses JWT (JSON Web Token) authentication to handle API requests securely.

- **Double-Token Security:** Upon authenticating via the login portal, the client receives a short-lived **Access Token** (15-minute TTL) and a secure **Refresh Token** (7-day TTL).
- **HttpOnly Cookie Storage:** The refresh token is stored inside an `HttpOnly`, `Secure`, and `SameSite=Strict` cookie, rendering it inaccessible to cross-site scripting (XSS) attacks.
- **Token Versioning (Forced Logout):** The `User` table includes a `tokenVersion` integer. Every token payload contains the version matching the database. If a user changes their password or an administrator terminates a session, the `tokenVersion` is incremented, immediately invalidating all active client tokens.
- **Argon2 Hashing:** User passwords are encrypted using the Argon2 hashing algorithm with customized parameters to withstand GPU-based dictionary attacks.

8.2 Role-Based Access Control (RBAC) Scopes

Permissions are not merely binary checks (e.g., ``is_admin``). Instead, the platform implements a granular **Scope Matrix** that determines which rows a user can access, modify, or delete:

Scope Level	Target Roles	Permissions and Access Boundaries
OWN	Student, Parent	Can access records explicitly linked to their own profile ID. Blocked from all other student rows.
ASSIGNED	Teacher, FYC	Can view/modify marks and attendance only for subjects allocated to them in the timetable.
DEPARTMENT	Department Admin	Can oversee and lock all student, teacher, and subject records belonging to their department.
ALL	Super Admin, COE	Can view, execute processes, and lock configurations college-wide (within their tenant).

8.0 Security Architecture (Cont.)

8.3 Data & API Protection Measures

The Express 5 backend contains multiple security layers designed to prevent standard web exploits:

- **Input Sanitisation (XSS Prevention):** A global sanitization middleware processes all incoming HTTP request bodies. It strip-filters HTML tags and potential script injections using the `js-xss` library.
- **SQL Injection Guard:** All database queries are executed using Prisma's parameterized query builder, making it impossible to perform SQL injection attacks.
- **Endpoint-Specific Rate Limiting:** In addition to a global rate limit of 500 requests per 15 minutes, sensitive routes (like `/api/auth/login` and password resets) use Redis-backed rate limiters to lock out IPs after 5 failed attempts.
- **HTTP Security Headers:** The server configures the `Helmet` package to append strict headers:
 - **HTTP Strict Transport Security (HSTS):** Enforces HTTPS connections for 2 years (includes subdomains).
 - **Content Security Policy (CSP):** Restricts script execution sources and blocks inline scripts.
 - **X-Frame-Options:** Set to `DENY` to prevent clickjacking attacks in frames.

8.4 Database-Level Guard Operations

Academic Ops secures critical business transactions using database constraints and lifecycle checks:

Semester Write-Lock Guard: A helper utility (`semester-lock.ts`) is called inside every marks and attendance service. It verifies the status of the associated academic semester. If the semester is set to `CLOSED` or `ARCHIVED` , the database transaction is aborted, preventing teachers or admins from editing historical records.

Role Lock Trigger: A database trigger defined on the `User` table prevents direct modifications to user roles. If an administrative query attempts to run `UPDATE "User" SET role = 'COE' WHERE id = X` , the database raises an exception unless verified by specific migration checks.

9.0 Scalability and Performance

9.1 Redis Caching & Graceful Degradation

To handle high-traffic periods, such as when results are published or attendance is finalized, Academic Ops uses a high-performance Redis cache layer:

- **API Response Caching:** Frequently accessed, read-heavy API responses (e.g. subject catalogues, timetables, active program configurations) are cached in Redis. Subsequent requests are served in under 5ms.
- **Graceful Degradation Fallback:** If the Redis instance (e.g., Upstash) goes offline, the `cache.service.ts` module automatically intercepts the connection failure. It switches the system to local, in-memory caching and continues API operations without crashing the server.

9.2 PgBouncer Connection Pooling

Serverless backend hosting plans or dynamic microservices can quickly exhaust database connections. Academic Ops prevents this by using PgBouncer:

- **Transaction Mode:** PgBouncer is configured in transaction pooling mode. This allows the backend to share a small pool of database connections across hundreds of active Express requests.
- **Prisma Configuration:** The backend configuration maintains two database URLs: a pooled URL (port 6543) for API operations and a direct URL (port 5432) for running migrations.

9.3 Database Indexing Strategy

Prisma migrations define more than 50 composite and single-column indexes on high-frequency tables. These include `idx_attendance_student_subject`, `idx_marks_student_subject`, and `idx_audit_logs_tenant`. As a result, query lookup times remain flat even as database tables scale to millions of rows.

Pagination Enforcement: The repository layer enforces pagination on all API lists. No endpoint can fetch unbounded arrays of student records. This keeps memory usage low and response payloads lightweight.

10.0 Deployment and DevOps

10.1 Prerequisites

- **Node.js:** Version 20.0.0 or higher.
- **PostgreSQL:** Version 16 or higher (compatible with Supabase, Neon, or RDS).
- **Redis:** Version 7 or higher.
- **Package Manager:** npm version 9 or higher (with workspaces support).

10.2 Local Development Workflow

The project utilizes npm workspaces to manage the monorepo structure. Developers can spin up the environment with a single terminal session:

```
# Install monorepo dependencies (client + server)
npm install

# Generate the type-safe Prisma client
cd server && npx prisma generate && cd ..

# Run database migrations
cd server && npx prisma migrate dev && cd ..

# Seed mock database with development profiles (optional)
cd server && npx tsx prisma/seed.ts && cd ..

# Start Next.js client (port 3001) and Express server (port 4000)
npm run dev
```

10.3 Docker Containerization

The repository contains multi-stage, production-ready Dockerfiles for both services. It also includes a `docker-compose.yml` file configured to spin up the entire application stack:

Service Name	Container Base Image	Port Mapping	Volume / Storage
postgres	postgres:16-alpine	5432:5432	postgres_data (persistent volume)
redis	redis:7-alpine	6379:6379	redis_data (AOF persistence)
server	node:20-alpine (Multi-stage)	4000:4000	Binds server/ for hot-reload in dev
client	node:20-alpine (Multi-stage)	3001:3000	Statically optimized for build distribution

10.0 Deployment and DevOps (Cont.)

10.4 Cloud Deployment and Infrastructure as Code

Academic Ops is pre-configured for automated cloud deployment via Render and Netlify:

- **Backend API (Render):** The `render.yaml` blueprint configures a web service named `acadops-api` in the Singapore region. It runs automatic database migrations (using a production script) during the build stage before launching the Express process.
- **Frontend Client (Netlify):** The `netlify.toml` file defines the configuration for hosting the Next.js client. It installs dependencies, builds the Next.js static files, and loads the `@netlify/plugin-nextjs` package to handle Next.js App Router API routes, SSR operations, and edge middleware.

10.5 Environment Configurations

Env Variable	Scope	Fallback / Default	Functional Purpose
DATABASE_URL	Server	—	Prisma PostgreSQL pooled connection string (PgBouncer port 6543).
DIRECT_DATABASE_URL	Server	—	Direct PostgreSQL database connection string (port 5432) for running migrations.
REDIS_URL	Server	redis://localhost:6379	Redis connection string for session tracking, rate limiting, and BullMQ workers.
JWT_SECRET	Server	—	Minimum 32-character key used to sign client Access Tokens.
SMTP_HOST / PORT	Server	smtp.gmail.com / 587	SMTP server host credentials for automated email dispatches.
BREVO_API_KEY	Server	—	Cloud email provider HTTP API key used as a high-volume fallback to SMTP.
FAST2SMS_API_KEY	Server	—	SMS gateway API key used to deliver absence notifications to parent phone numbers.
NEXT_PUBLIC_API_URL	Client	http://localhost:4000/api	Base path URL pointing Next.js API requests to the active Express server.

11.0 Project Structure

The monorepo structure segregates the frontend and backend codes into separate workspaces. This architecture makes it easy for developers to write, test, and deploy features in isolation:

Directory / File Path	Role and Functional Component Details
client/src/app/	Next.js App Router containing route segments for role-based dashboards (/dashboard/student , /dashboard/teacher , /dashboard/admin).
client/src/components/ui/	Reusable design primitives (buttons, badges, inputs, card tiles) built using Tailwind CSS.
client/src/lib/api.ts	Axios client config with interceptors for token attachment and automatic refresh flows.
client/src/lib/auth-store.ts	Zustand store managing local authentication state and active token version records.
server/src/api/routes/	Contains 29 Express routing modules that handle API routing for specific sub-systems (e.g. admissions.routes.ts).
server/src/api/middleware/	Secures HTTP routes. Includes auth checking, rate limiting, and tenant scoping.
server/src/services/	Business logic layer containing 50+ service classes (e.g. internal-assessment.service.ts) handling transactions.
server/src/data-access/	Repository layers containing 20+ query definitions that interact with Prisma Client APIs.
server/prisma/schema.prisma	Main database schema defining 40+ models, indexes, and relational links.
server/prisma/migrations/	26+ database migration files tracking development schema changes.

12.0 Compliance and Audit

12.1 Data Retention Policy

To meet educational regulations (including UGC, AICTE, and university guidelines), Academic Ops implements a strict data retention and storage configuration:

Data Class	Retention Period	Storage Medium / Compliance Standard
Student Profiles	Indefinite (Alumni Archive)	PostgreSQL DB (active) / S3 Glacier (archived after 10 years).
Academic Grades	Indefinite (Official Records)	Strictly logged and persisted within primary PostgreSQL tables.
Attendance Sheets	6 Years (Or graduation + 2 years)	Preserved for university audits. Archiving can be triggered by Super Admin.
Audit logs	Indefinite (Audit verification)	Kept in a secure table. Triggers alert administrators if record count exceeds 100k.
Email / SMS logs	1 Year	Used for dispute resolution. Automatically pruned after 1 year.

12.2 Immutable System Audit Capabilities

The audit tracking module (`audit-log.repository.ts`) records every critical change to the database:

- **Diff Logging:** Every insert, update, or delete operation logs the **User ID, Timestamp, IP Address, User Agent**, and the exact **Old and New JSON payloads** of the changed fields.
- **Immutable Design:** The `AuditLog` database table has no corresponding update or delete API routes, protecting logs from tampering.

12.3 Separation of Duties Compliance

Academic workflows follow strict checks and balances. For example, Clerks can upload grades but cannot finalize results. Teachers enter student attendance, but only Department Admins can authorize corrections to locked sheets. Mentees' scores must be verified by their Mentor before final submission, preventing academic fraud.

13.0 Key Benefits & 14.0 Contributing

13.1 Key Benefits by Stakeholder

Stakeholder	Core Operational Benefits Provided
Students	Instant access to attendance history and IA marks. View final exam results and timetables securely from their mobile browser.
Teachers	Automated IA best-of-N calculations. Batch attendance and marks uploads via CSV. Secure request workflow to edit locked registers.
HODs & Admins	Real-time oversight of department operations. Instant dashboard metrics. Centralized mentor assignment and student tracking.
Institutions	Unified multi-tenant hosting with full data separation. Lower administrative costs, zero paper waste, and secure audit trails.

14.0 Contributing Workflow

To maintain code quality and database consistency, the development team follows a structured contribution pipeline:

- 1. Branch Strategy:** Developers fork the main repository and create feature branches following the naming convention `feature/[feature-name]` or `bugfix/[issue-name]`.
- 2. Local Linting & Styling:** Pre-commit hooks run ESLint checks on the Next.js client code and trigger unit tests on the Express backend using Vitest.
- 3. Pull Requests:** Code changes are submitted via a Pull Request (PR) to the `main` branch. The PR must link to a valid development task.
- 4. CI/CD Integration:** GitHub Actions triggers on every PR push. It boots a PostgreSQL test container, runs Prisma database migrations, runs frontend lints, builds the client and server code, and executes all test suites.

15.0 Conclusion

The **Academic Ops Platform** represents a complete, secure, and production-ready solution for managing the academic operations of engineering colleges. By combining Next.js 16 and Express 5, the platform delivers a fast, responsive user interface alongside a reliable backend.

Key highlights of the platform's architectural design include:

- **Strict Multi-Tenant Isolation:** Secured at the application level via database query scoping, ensuring complete privacy across multiple client institutions.
- **Automated Academic Calculations:** Automates core tasks, such as student cycle swapping, IA best-of-N computations, and GPA calculations. This reduces administrative overhead and minimizes manual errors.
- **Comprehensive Security Controls:** Implements JWT session management, token versioning, password hashing, and Helmet headers. The system stands protected against common web security exploits.
- **Immutable Audit Trails:** Captures detailed old/new values, IP addresses, and user-agent metadata. The platform is ready for institutional compliance and verification audits.
- **Scalable and Responsive Architecture:** Uses Redis caching, PgBouncer pooling, and BullMQ background workers to keep response times fast, even during peak exam result publications.

The system is fully documented, statically tested, and containerized for direct deployment. This makes it an ideal choice for institutional rollouts, academic management upgrades, or further custom feature developments.

Detailed Project Report Final Sign-Off

Department Head (CSE)

Controller of Examinations